

Angular → Nx + Module Federation Migration Guide

Angular 19.1 · Node 20.17 · TS 5.7 · npm · Linux deployment target

One correction up front: **Nx moved Module Federation out of @nx/angular into its own @nx/module-federation package** (as of Nx ~19.5+, with the @nx/angular/module-federation re-export path fully removed by Nx v23). Use @nx/module-federation for ModuleFederationConfig, withModuleFederation, etc. Under the hood Nx uses @module-federation/enhanced (Zack Jackson / ByteDance), not raw Webpack ModuleFederationPlugin.

1. Workspace Setup

```
npx create-nx-workspace@latest eqbeam-mfe --preset=apps --pm=npm
cd eqbeam-mfe
npx nx add @nx/angular
npx nx add @nx/module-federation
```

Generate host + remotes in one shot:

```
npx nx g @nx/angular:host apps/shell --remotes=pending-orders,order-blotter
```

This scaffolds `shell`, `pending-orders`, `order-blotter`, each with `module-federation.config.ts`, a `bootstrap.ts`, and `build/serve` targets already switched to the `webpack-based` executors (`@nx/angular:webpack-browser` + `@nx/module-federation:module-federation-dev-server`).

Add a remote later and auto-wire it to the shell:

```
npx nx g @nx/angular:remote apps/watchlist --host=shell
```

Folder structure

```
eqbeam-mfe/
├─ apps/
│   └─ shell/                                # host/container
```

```

| | | └─ src/app/
| | | └─ src/bootstrap.ts
| | | └─ src/main.ts
| | | └─ module-federation.config.ts
| | |   └─ module-federation.manifest.json    # dynamic remote URLs per env
| | └─ pending-orders/                      # remote MFE 1
| |   | └─ src/app/remote-entry/            # what actually gets exposed
| |   |   └─ module-federation.config.ts
| | └─ order-blotter/                      # remote MFE 2
└─ libs/
  | └─ shared/
  |   | └─ ui/                             # dumb/presentational components
  |   |   | └─ status-chip/
  |   |   |   └─ order-row/
  |   | └─ data-access/                    # facades, HTTP clients, RSocket wrappers
  |   |   | └─ auth/
  |   |   |   └─ orders-api/
  |   | └─ util/                          # pure functions, formatters, types
  |   |   └─ feature/                      # composed feature libs (facade + smart compon
  |   |       └─ pending-orders/
  | └─ eqbeam-design-tokens/              # scss variables/mixins shared across MFES
└─ nx.json
└─ package.json

```

Naming convention: `libs/<scope>/<type>/<name>` — mirrors your existing three-layer preference (API service → facade → component). `data-access` = API/service layer, `feature` = facade + smart component, `ui` = presentational only. Keep business logic out of `apps/*`; apps should be thin composition roots, same discipline you already enforce in `BeamAgGridComponent` -adjacent code.

2. Migration Strategy: Incremental, Not Big Bang

For a mature app already on standalone components, big-bang is unnecessary risk. Recommended path:

1. **Scaffold the Nx workspace fresh** (above), don't try to convert your existing repo's folder layout in place.
2. **Move code, don't rewrite it.** Use `git mv` or plain copy for your existing `src/app/**` into a new `apps/shell/src/app/` (or directly into `libs/` if it's reusable). Standalone components port over almost unchanged — no NgModule untangling needed, which is the single biggest reason this migration is low-risk for you.

3. **Get the shell building and serving as a monolith first**, inside Nx, with zero federation. Confirm parity (routing, AG Grid, RSocket streaming, logging lib) before touching Module Federation config.
4. **Peel off one remote at a time**, starting with your lowest-risk, most self-contained component — good candidate: something like `PendingOrders` that has no child routes and a narrow dependency surface. Expose it, consume it back in the shell via Module Federation (not a normal import), verify behavior is identical, then move to the next.
5. **Extract shared libs opportunistically** as you find duplication between shell and remotes — don't pre-extract everything on day one; that's how you get premature abstraction and circular deps.
6. Only after 2–3 remotes are stable do you tackle cross-cutting concerns: shared auth state, design tokens, versioning policy.

This lets you keep shipping from the shell throughout, which matters if EQBEAM has active trading-desk users during the migration window.

3. Module Federation Configuration

Host (`apps/shell/module-federation.config.ts`)

```
import { ModuleFederationConfig } from '@nx/module-federation';

const config: ModuleFederationConfig = {
  name: 'shell',
  remotes: ['pending-orders', 'order-blotter'],
};

export default config;
```

For production you don't want remote URLs baked in at build time — use **Dynamic Module Federation** with a manifest so each remote can deploy and version independently.

`apps/shell/src/main.ts`:

```
import { setRemoteDefinitions } from '@nx/module-federation';

fetch('/module-federation.manifest.json')
  .then((res) => res.json())
  .then((definitions) => setRemoteDefinitions(definitions))
  .then(() => import('./bootstrap'))
```

```
.catch((err) => console.error('Failed to load remote definitions', err));
```

apps/shell/module-federation.manifest.json (swap per environment at deploy time, not build time):

```
{
  "pending-orders": "http://localhost:4201/remoteEntry.js",
  "order-blotter": "http://localhost:4202/remoteEntry.js"
}
```

In prod this becomes something like `https://mfe-pending-orders.eqbeam.internal/remoteEntry.js`, swapped via your deployment pipeline/config service without rebuilding the shell.

Once you're on dynamic federation, remove the static `remotes: [...]` array from the host's `module-federation.config.ts`.

Remote (apps/pending-orders/module-federation.config.ts)

```
import { ModuleFederationConfig } from '@nx/module-federation';

const config: ModuleFederationConfig = {
  name: 'pending-orders',
  exposes: {
    './PendingOrdersComponent': 'apps/pending-orders/src/app/remote-entry/entry.t
  },
};

export default config;
```

Key point for your "don't load the whole app" requirement: **only expose what's under exposes**. Everything else in the remote's `src/app` is just implementation detail — it never gets pulled into the host unless it's imported by the exposed entry file's dependency graph. This is what actually solves your current problem (host previously loading the whole remote app instead of a single component).

Shared dependency config (avoiding version conflicts)

Nx auto-detects and shares npm + workspace deps as singletons based on the project graph, but for a trading app pulling in AG Grid, RxJS, and RSocket, be explicit rather than relying purely on auto-detection:

```
import { ModuleFederationConfig, SharedFunction } from '@nx/module-federation';
```

```

const shareConfig: SharedFunction = (libraryName, sharedConfig) => {
  if (libraryName === 'ag-grid-angular' || libraryName === 'ag-grid-enterprise')
    return { ...sharedConfig, singleton: true, strictVersion: true };
}
if (libraryName === 'rxjs') {
  return { ...sharedConfig, singleton: true, requiredVersion: 'auto' };
}
return sharedConfig; // fall back to Nx defaults for everything else
};

const config: ModuleFederationConfig = {
  name: 'pending-orders',
  exposes: {
    './PendingOrdersComponent': 'apps/pending-orders/src/app/remote-entry/entry.t
  },
  shared: shareConfig,
};

export default config;

```

`singleton: true + strictVersion: true` on AG Grid Enterprise matters because you're licensing it once and you do **not** want two AG Grid instances/CSS registrations fighting for the same license key or theme registry across host and remote.

4. Worked Example: Exposing `PendingOrders`

Given: no child routes, depends on `AuthService`, uses RxJS, styled with SCSS.

`apps/pending-orders/src/app/remote-entry/entry.ts` — this is the actual federated module, kept intentionally thin:

```

export { PendingOrdersComponent } from '../pending-orders/pending-orders.componen

```

`libs/shared/data-access/auth/src/lib/auth.service.ts` — move `AuthService` into a shared lib so both shell and remote resolve the *same* singleton instance rather than each bootstrapping their own:

```

import { Injectable, signal } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly _token = signal<string | null>(null);

```

```

    readonly token = this._token.asReadonly();

    setToken(token: string): void {
        this._token.set(token);
    }
}

```

libs/shared/feature/pending-orders/src/lib/pending-orders.component.ts (or keep it in the remote app if it's genuinely not reused elsewhere — don't force everything into `libs/` prematurely):

```

import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { AuthService } from '@eqbeam/shared/data-access/auth';
import { OrdersApiService } from '@eqbeam/shared/data-access/orders-api';

@Component({
    selector: 'eqb-pending-orders',
    standalone: true,
    imports: [CommonModule],
    templateUrl: './pending-orders.component.html',
    styleUrls: ['./pending-orders.component.scss'],
})
export class PendingOrdersComponent {
    private readonly auth = inject(AuthService);
    private readonly ordersApi = inject(OrdersApiService);

    readonly orders = this.ordersApi.pendingOrders; // signal from facade
}

```

Because `AuthService` is `providedIn: 'root'` **and** lives in a shared lib that's marked `singleton: true` in the shared config, both shell and remote inject the same instance at runtime — no duplicated auth state, no re-login inside the remote.

Styling: SCSS is compiled at build time per-project, so there's no runtime coupling — but *tokens* (colors, spacing, your dark-terminal palette, IBM Plex Mono) should live in `libs/eqbeam-design-tokens` and be `@use` 'd by both shell and remote SCSS entry points, so a BUY/SELL accent color or status-chip color only has one source of truth.

Consuming it in the shell, lazy-loaded, no routing required since it has no child routes — load it directly into a container:

```

import { Component, ViewContainerRef, inject, viewChild } from '@angular/core';

```

```

@Component({
  selector: 'app-dashboard-panel',
  standalone: true,
  template: `<ng-container #host></ng-container>`,
})
export class DashboardPanelComponent {
  private readonly host = viewChild.required('host', { read: ViewContainerRef });

  async ngAfterViewInit() {
    const m = await import('pending-orders/PendingOrdersComponent');
    this.host().createComponent(m.PendingOrdersComponent);
  }
}

```

If it *did* need routing, you'd expose `./Routes` instead of a component and consume via

```
loadChildren: () => import('pending-orders/Routes').then(m => m.remoteRoutes) .
```

4b. Dual Exposure: Your Own Shell vs. a Fully External App

`eqbeam-options` needs to consume its own remotes (`pending-orders`, `order-blotter`, etc.) **and** a completely separate, independently-deployed app (own domain, own login/session) needs to consume the same remote. These are different trust and runtime boundaries, so expose the component two ways from the same source:

```

apps/pending-orders/src/app/remote-entry/
├─ entry.ts      # Module Federation entry – for your own shell, same workspace
└─ element.ts    # Angular custom element entry – for fully external consumers

```

entry.ts (unchanged from section 4) — plain re-export, consumed via `import('pending-orders/PendingOrdersComponent')`, resolves shared singletons (`AuthService`, `RSocket` connection) from the host's injector because both live on the same page in the same Nx workspace.

element.ts — self-contained custom element, its own Angular platform/injector, no assumption about the consuming app's DI tree, version, or session:

```

import { createApplication } from '@angular/platform-browser';
import { createCustomElement } from '@angular/elements';
import { PendingOrdersComponent } from '../pending-orders/pending-orders.component';

(async () => {
  const app = await createApplication({ providers: [] });

```

```

customElements.define(
  'eqb-pending-orders',
  createCustomElement(PendingOrdersComponent, { injector: app.injector })
);
})();

```

Since the external app can't resolve your shared `AuthService`, the component takes `auth/config` as `@Input()` s instead of via DI when running in element mode:

```

@Component({ selector: 'eqb-pending-orders', standalone: true, ... })
export class PendingOrdersComponent {
  @Input() authToken?: string;      // set by external consumer; ignored when run
  @Input() apiBaseUrl?: string;
}

```

Add a second build target (`apps/pending-orders/project.json`) that bundles `element.ts` standalone (no Module Federation shared-scope negotiation) so it can be dropped in via a plain `<script>` tag + CORS, independent of whatever Module Federation setup (if any) the external app runs.

Type safety across this boundary won't come from Nx's generated federation `.d.ts` — that only works inside your workspace. Hand-maintain (or publish as a small scoped package, e.g. `@eqbeam/pending-orders-contract`) a `.d.ts` describing the element's attributes/events as the versioned public contract for external consumers.

4c. RSocket Connections: Shared vs. Per-MFE

Two different strategies apply depending on which exposure path is in play — don't try to force one connection-sharing model to cover both:

Inside your own workspace (shell + its remotes, same page): one RSocket connection, shared singleton, resolved via DI — this is what section 4/"Common Pitfalls" already covers (`singleton: true, strictVersion: true, eager: true` in the shared config, service in a shared lib, `providedIn: 'root'`). Sharing here is correct: it's the same session, same auth, same page.

When consumed by the fully external app: do **not** try to share the connection — establish a fresh, independent RSocket connection scoped to that MFE instance instead. Concretely:

- The custom-element entry (`element.ts`) should construct its own RSocket connector using the `apiBaseUrl` / `authToken` passed in as inputs, rather than injecting the shared connection service used by your own shell.

- Wire connection lifecycle to the element's own lifecycle (`connectedCallback` / `disconnectedCallback` via `ngOnInit` / `ngOnDestroy` in the wrapped component) so the connection opens when the external app mounts it and closes cleanly when it's removed — don't rely on the shell's app-lifetime connection management, since there is no shell in that context.
- This means the *same* `PendingOrdersComponent` needs its data layer to accept either an injected shared connection (workspace mode) or a locally-constructed one (element mode). Cleanest way to do that without branching inside the component itself: define an `OrdersConnectionToken` (`InjectionToken`) that the workspace entry (`entry.ts`) binds to the shared singleton, and that the element entry (`element.ts`) binds to a per-instance connection built from the `@Input()` s — the component just injects the token and stays agnostic to which mode it's running in.

```
// libs/shared/data-access/orders-api/src/lib/orders-connection.token.ts
import { InjectionToken } from '@angular/core';
import type { RSocket } from 'rsocket-core';

export const ORDERS_CONNECTION = new InjectionToken<RSocket>('ORDERS_CONNECTION')

// entry.ts (workspace mode) — binds to the existing shared singleton
providers: [{ provide: ORDERS_CONNECTION, useExisting: SharedOrdersConnectionServ

// element.ts (external mode) — builds a fresh, per-instance connection
const connection = buildOrdersConnection({ apiBaseUrl, authToken });
providers: [{ provide: ORDERS_CONNECTION, useValue: connection }]
```

This keeps the “one connection per MFE instance when external, one shared connection when internal” split explicit and testable, instead of it happening accidentally.

5. Type Safety Across the Federation Boundary

Nx's Module Federation support generates a `.d.ts` federation type map so

`import('pending-orders/PendingOrdersComponent')` is typed instead of `any`, provided both projects are in the same Nx workspace (this is one of the strongest arguments for the monorepo approach over publishing each MFE as a separate repo). Run:

```
npx nx run-many --target=build --projects=pending-orders,order-blotter
```

before serving the shell in isolation, so the manifest/type declarations exist. In CI, build remotes before the host for the same reason.

6. Shared State Across MFEs

Given your existing preference for Signals over NgRx: keep it that way here too — it scales fine across federation boundaries as long as the *service holding the signal* is a true singleton (shared lib, `providedIn: 'root', singleton: true` in shared config). Avoid a separate cross-MFE state library with its own event bus unless you have remotes that are deployed and versioned completely independently and can't share a lib version — that's the one case where signals-in-a-shared-lib breaks down and you'd reach for a runtime message channel (e.g. `BroadcastChannel`, or your existing RSocket connection acting as the source of truth instead of client-side shared state).

7. Dev, Build, Deploy

Dev — serve host + live remotes:

```
npx nx serve shell --devRemotes=pending-orders,order-blotter
```

Dev — work on one remote in isolation:

```
npx nx serve pending-orders
```

Build everything (full redeploy):

```
npx nx run-many --target=build --all
```

Build only what changed (affected):

```
npx nx affected --target=build
```

Deployment on your Linux target: each app (`shell`, `pending-orders`, `order-blotter`) builds to its own `dist/apps/<name>` and can be deployed as an independent static bundle behind your web server/CDN — that's the actual payoff of this whole migration. Only the shell's `module-federation.manifest.json` needs to point at wherever each remote's `remoteEntry.js` currently lives; swap that file per environment (dev/staging/prod) rather than rebuilding.

8. Versioning Shared Libraries

- Version the **shared libs**, not the apps, if you ever publish them to a private npm registry for consumption outside this Nx workspace. Inside the workspace, Nx's project graph + `nx affected` is your versioning mechanism — no manual semver needed as long as everything stays in one repo.
- If/when a remote needs to be deployed and consumed by a team **outside** this workspace, that's the trigger to actually publish `libs/shared/*` as versioned npm packages with real semver and a changelog — don't do this preemptively.
- Coordinate Angular/AG Grid major version bumps explicitly across all remotes; Module Federation's singleton sharing means a version mismatch degrades to duplicate copies loaded at runtime (silent perf/bugs) rather than a hard failure, which is the most common MFE pitfall.

9. Common Pitfalls

- **Exposing whole modules instead of leaf components** — recreates your current problem inside the new architecture. Keep `exposes` narrow.
- **Non-singleton shared libs** — if `AuthService` or your RSocket connection service isn't a true shared singleton, each remote opens its own connection/auth state, which for a streaming trading app is both a resource leak and a correctness bug (duplicate subscriptions, stale tokens).
- **CSS leaking across remotes** — AG Grid theme classes and global resets can collide if remotes don't scope their styles or share design tokens consistently. Centralize tokens now, before you have 4+ remotes to retrofit.
- **Building host before remotes in CI** — breaks the generated type map; always build remotes first (or use `run-many` so Nx sequences by the dependency graph).
- **Skipping the "serve as one app first" step** — federation bugs are much harder to diagnose than ordinary Angular bugs; don't introduce both a monorepo migration and Module Federation risk simultaneously.

10. Testing Strategy

- **Unit/component tests:** unchanged — Jasmine + Karma per-project, same as your current setup (`ng-jasmine-kit` / `AgGridHarness` pattern ports over per-lib without modification).
- **Federation integration tests:** serve host + all live remotes together (`nx serve shell -devRemotes=...`) as a smoke-test target in CI — Playwright or Cypress hitting the shell and asserting the remote actually mounted, not just that it built.
- **Contract tests for exposed components:** since a remote's exposed component is effectively a public API now, treat its `@Input` / `@Output` (or signal-based equivalents) as a contract — a dedicated spec per exposed entry file that fails if the public surface changes without an explicit version bump.

Quick command reference

Task	Command
Create workspace	<code>npm create-nx-workspace@latest eqbeam-mfe --preset=apps</code>
Add Angular + MF plugins	<code>npm nx add @nx/angular && npm nx add @nx/module-federation</code>
Generate host + remotes	<code>npm nx g @nx/angular:host apps/shell --remotes=pending-orders,order-blotter</code>
Add another remote	<code>npm nx g @nx/angular:remote apps/watchlist --host=shell</code>
Generate shared lib	<code>npm nx g @nx/angular:lib libs/shared/data-access/auth</code>
Dev serve all	<code>npm nx serve shell --devRemotes=pending-orders,order-blotter</code>
Build affected	<code>npm nx affected --target=build</code>
Build all	<code>npm nx run-many --target=build --all</code>